Laboratorio 4 de Simulación Computacional

Generación de variables aleatorias continuas

Universidad	Universidad de los Llanos, Escuela de Ingeniería
Curso	Simulación Computacional
Docente	Angel Alfonso Cruz Roa
Estudiante	Luciana Alejandra Belandria Agudelo
Código	160005005

2. Consulta previa

- **¿Qué es una variable aleatoria?**
  Es una cantidad cuyo valor depende del azar y que no se conoce hasta que el experimento ocurre. La describe su función de distribución $F(x) = P\{X \leq x\}$, que dice qué tan probable es que la variable no pase de cierto valor x.

- **¿Qué es una distribución de probabilidad?**
  Es la regla que reparte la probabilidad entre los valores que puede tomar una variable aleatoria. Se expresa con la función de distribución F, que es creciente, y en el caso continuo también con una función de densidad f, de manera que $F(x) = \int_{-\infty}^{x} f(y)\,dy$.

- **¿Cuál es la diferencia entre una distribución de probabilidad discreta y una distribución de probabilidad continua?**
  En la discreta cada valor puntual tiene su propia probabilidad y se usa una función de masa. En la continua se usa una densidad y las probabilidades se obtienen integrándola sobre intervalos.

- **¿Cuál es la definición de la distribución de probabilidad exponencial?**
  Una variable exponencial con tasa λ y media $1/\lambda$ tiene función de distribución y densidad

$$F(x) = 1 - e^{-\lambda x}, \qquad f(x) = \lambda e^{-\lambda x}, \qquad x > 0.$$

- **¿Cuál es la definición de la distribución de probabilidad Gamma y sus parámetros?**
  Una variable Gamma con parámetros (n, λ) tiene función de distribución

$$F(x) = \int_0^x \frac{\lambda e^{-\lambda y}(\lambda y)^{n-1}}{(n-1)!}\,dy.$$

  En general, con parámetro de forma $\alpha > 0$ y tasa $\lambda > 0$, su densidad es $f(x) = Ke^{-\lambda x}x^{\alpha-1}$ para $x > 0$, con $K = \lambda^{\alpha}/\Gamma(\alpha)$, y su media es α/λ. Una Gamma (n, λ) equivale a la suma de n exponenciales independientes de tasa λ.

- **¿Cuál es la definición de la distribución de probabilidad normal estándar y sus parámetros?**
  Es la normal con media $\mu = 0$ y varianza $\sigma^2 = 1$, con densidad

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}, \qquad -\infty < x < \infty.$$

  Si Z es normal estándar, entonces $\mu + \sigma Z$ es normal con media μ y varianza σ^2.

- **¿Qué es un proceso de Poisson y un proceso de Poisson no homogéneo? ¿Cuál es la diferencia entre ellos?**
  Un proceso de Poisson con tasa λ cuenta eventos en el tiempo, y los tiempos entre eventos sucesivos son exponenciales independientes con tasa λ. Tiene incrementos independientes y estacionarios, y el número de eventos en un intervalo de longitud h es Poisson con media λh. Un proceso de Poisson no homogéneo relaja el supuesto de incrementos estacionarios, de modo que la tasa cambia con el tiempo según una función de intensidad $\lambda(t)$. La diferencia es entonces que en el homogéneo la tasa es constante y en el no homogéneo varía con el tiempo.

5. Procedimiento o metodología

Los números uniformes se generaron con el generador congruencial mixto $X_{n+1} = (aX_n + c)$ mód m, $u_i = X_i/m$, con $X_0 = 2391$, $a = 25214903917$, $c = 11$ y $m = 2^{48}$. Se generó una secuencia de un millón de números con `genranN` y la función `nextU` la fue entregando en orden. En total se usaron 382 819 números.

```
def genran(a, c, m, xant):
    xsig = (a * xant + c) % m
    usig = xsig / m
    return [xsig, usig]

def genranN(a, c, m, x0, N):
    xant = x0
    I = []; X = []; U = []
    for t in range(1, N + 1):
        [xi, ui] = genran(a, c, m, xant)
        I.append(t); X.append(xi); U.append(ui)
        xant = xi
    return [I, X, U]

x0, a_g, c_g, m_g = 2391, 25214903917, 11, 2**48
I, X, U_seq = genranN(a_g, c_g, m_g, x0, 1000000)

pos = 0
def nextU():
    global pos
    u = U_seq[pos]
    pos += 1
    return u
```

Listing 1: Generador congruencial mixto

Los primeros diez números fueron 0.2142, 0.7450, 0.6610, 0.7595, 0.7835, 0.3924, 0.6607, 0.8984, 0.3532 y 0.7253. Para cumplir la nota de la guía se aplicaron a los primeros 10 000 números la prueba χ^2 de uniformidad y la prueba de rachas, con $\alpha = 0.05$. La tabla 1 muestra que el generador acepta las dos.

Prueba	Estadístico	Valor crítico	Resultado
χ^2 $(k = 10)$	10.6000	16.9190	Acepta uniformidad
Rachas $(R = 6699)$	$Z = 0.7748$	1.9600	Acepta aleatoriedad

Tabla 1: Pruebas sobre el generador congruencial mixto.

5.1. Generación de variables aleatorias continuas

En cada literal se generaron 1000 valores y se comparó su histograma con la densidad teórica.

a. $f(x) = e^x/(e-1),\ 0 \le x \le 1$

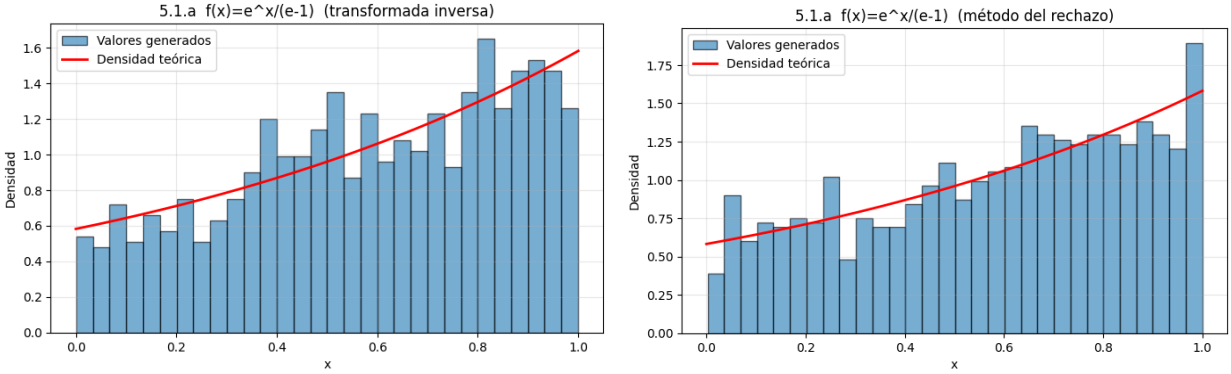
Por transformada inversa, $F(x) = (e^x - 1)/(e-1)$ y $X = \log\left(1 + (e-1)U\right)$. También se implementó el rechazo con $g(x) = 1$ en $(0, 1)$, donde $c = e/(e-1) \approx 1.582$ y se acepta $X = U_1$ si $U_2 \le e^{U_1 - 1}$.

```
def gen_a_inversa():
    return math.log(1 + (math.e - 1) * nextU())

def gen_a_rechazo():
    it = 0
    while True:
        it += 1
        u1, u2 = nextU(), nextU()
        if u2 <= math.exp(u1 - 1):
            return u1, it
```

Listing 2: Literal a

Con la transformada inversa la media muestral fue 0.5842 frente a una teórica de 0.5820, y los primeros valores fueron 0.3134, 0.8242, 0.7588, 0.8351 y 0.8528. Con el rechazo la media fue 0.5811 y se necesitaron en promedio 1.6030 iteraciones, muy cerca del c teórico de 1.5820. En la figura 1 ambos histogramas siguen la densidad teórica.



(a) Transformada inversa

(b) Rechazo

Figura 1: Literal a.

b. Densidad por tramos en $[2, 6]$

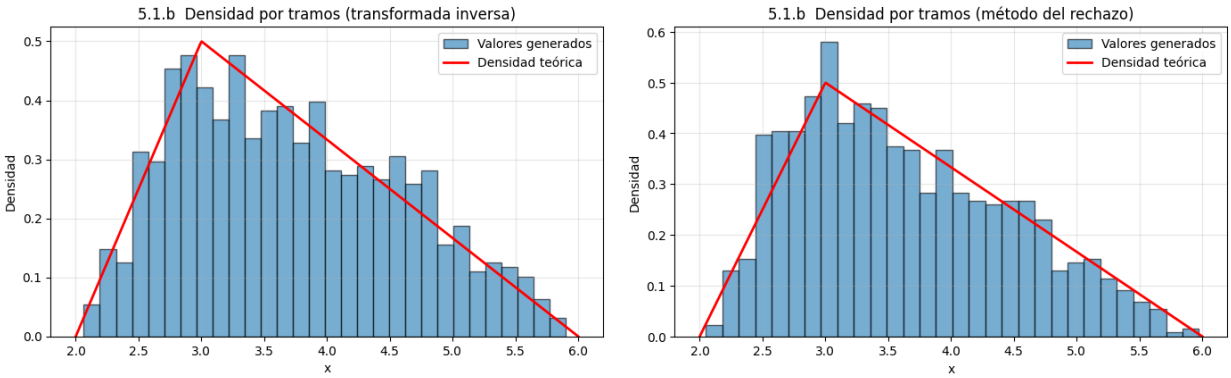
Integrando por tramos, $F(x) = (x-2)^2/4$ en $[2,3]$, con $F(3) = 1/4$, y $F(x) = x - x^2/12 - 2$ en $[3,6]$. Al invertir queda $X = 2+2\sqrt{U}$ si $U \leq 1/4$ y $X = 6-2\sqrt{3(1-U)}$ si $U > 1/4$. Para el rechazo se usó $g = 1/4$ en $(2,6)$, con $c = 2$, aceptando $Y = 2+4U_1$ si $U_2 \leq 2f(Y)$.

```
def gen_b_inversa():
    u = nextU()
    if u <= 0.25:
        return 2 + 2 * math.sqrt(u)
    return 6 - 2 * math.sqrt(3 * (1 - u))

def gen_b_rechazo():
    it = 0
    while True:
        it += 1
        y = 2 + 4 * nextU()
        if nextU() <= 2 * float(fb(y)):
            return y, it
```

Listing 3: Literal b

Con la transformada inversa la media fue 3.7178 frente a 3.6667, y los primeros valores fueron 3.7554, 4.5370, 3.5044, 4.5974 y 4.7748. Con el rechazo la media fue 3.6112 y el promedio de iteraciones 2.0110, frente al c teórico de 2 (figura 2).



(a) Transformada inversa

(b) Rechazo

Figura 2: Literal b.

c. $F(x) = (x^2+x)/2,\ 0 \leq x \leq 1$

Resolviendo $x^2 + x - 2U = 0$ y tomando la raíz positiva se obtiene $X = \left(-1+\sqrt{1+8U}\right)/2$, con densidad $f(x) = x + 1/2$.

```
def gen_c_inversa():
    return (-1 + math.sqrt(1 + 8 * nextU())) / 2
```

Listing 4: Literal c

La media muestral fue 0.5774 frente a una teórica de 0.5833, y los primeros valores fueron 0.8378, 0.0901, 0.7539, 0.6936 y 0.5896. El histograma sigue la recta $f(x) = x + 1/2$ (figura 3).

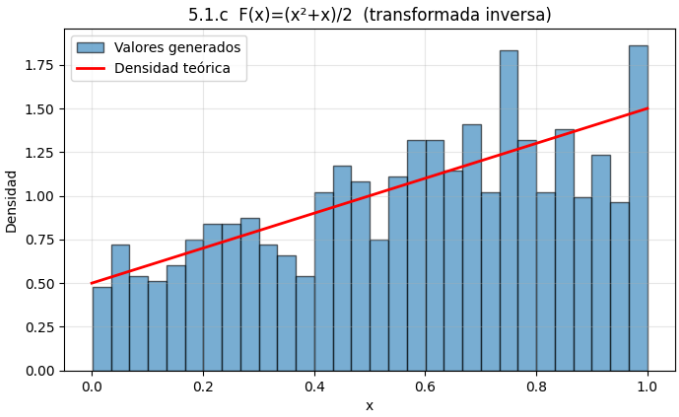


Figura 3: Literal c.

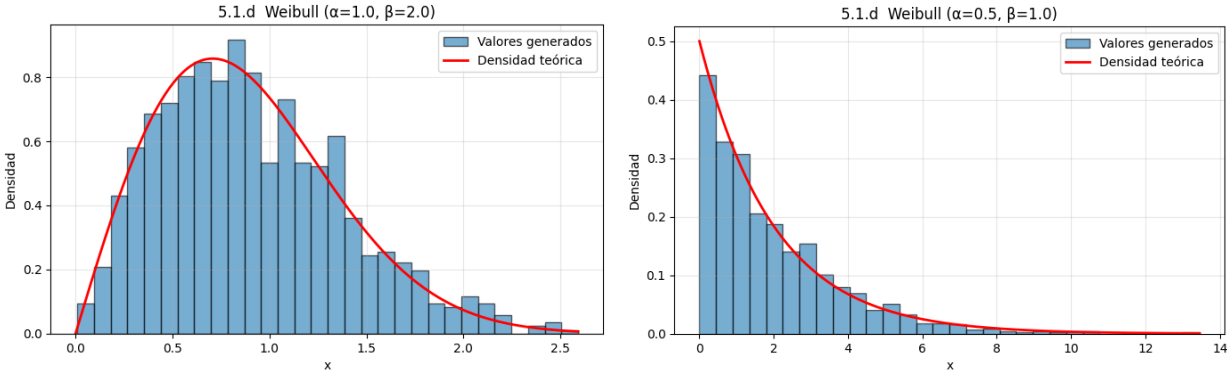
d. Weibull, $F(x) = 1 - \exp(-\alpha x^{\beta})$

Despejando se obtiene $X = \left(-\frac{1}{\alpha}\log U\right)^{1/\beta}$, con densidad $f(x) = \alpha\beta x^{\beta-1}e^{-\alpha x^{\beta}}$. Se usaron $\alpha = 1,\ \beta = 2$ y $\alpha = 0.5,\ \beta = 1$.

```
def gen_weibull(alpha, beta):
    return (-math.log(nextU()) / alpha) ** (1 / beta)
```

Listing 5: Literal d

Con $\alpha = 1$ y $\beta = 2$ la media muestral fue 0.9090, con primeros valores 0.9310, 0.4759, 0.5286, 0.8616 y 0.6452. Con $\alpha = 0.5$ y $\beta = 1$ la media fue 1.9790, con primeros valores 0.6846, 0.7164, 1.7337, 0.3709 y 2.8670. En ambos casos el histograma sigue la densidad teórica (figura 4).



(a) $\alpha = 1,\ \beta = 2$ (b) $\alpha = 0.5,\ \beta = 1$

Figura 4: Literal d.

5.2. Generando procesos de Poisson

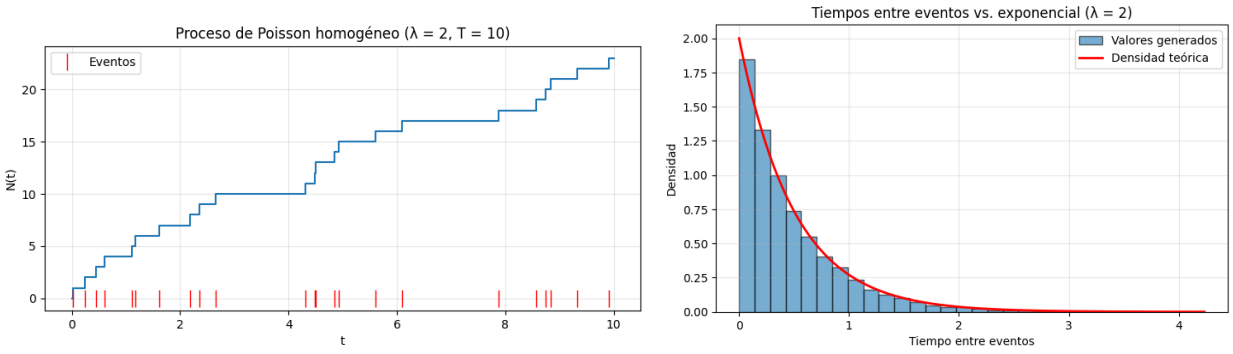
a. Primeras T unidades de tiempo de un proceso de Poisson con tasa λ

Se suman tiempos exponenciales $-\frac{1}{\lambda}\log U$ hasta que el tiempo acumulado supera T, registrando cada evento. Se usó $T = 10$ y $\lambda = 2$.

```
def proceso_poisson(T, lam):
    t, I, S = 0.0, 0, []
    while True:
        t = t - math.log(nextU()) / lam
        if t > T:
            break
        I += 1
        S.append(t)
    return I, S
```

Listing 6: Proceso de Poisson

En la corrida ocurrieron $I = 23$ eventos, con valor esperado $\lambda T = 20$. Los tiempos de los eventos fueron 0.0209, 0.2437, 0.4458, 0.6102, 1.1172, 1.1710, 1.6133, 2.1863, 2.3487, 2.6510, 4.3074, 4.4783, 4.4970, 4.8471, 4.9303, 5.5972, 6.0925, 7.8812, 8.5660, 8.7477, 8.8398, 9.3198 y 9.9051. Al repetir la simulación 1000 veces el promedio de $N(T)$ fue 19.9340, y los tiempos entre eventos se ajustaron a la exponencial de tasa 2 (figura 5).



(a) Realización de $N(t)$

(b) Tiempos entre eventos

Figura 5: Proceso de Poisson con $\lambda = 2$ y $T = 10$.

b. Variable aleatoria de Coxian

El algoritmo suma el tiempo de cada etapa, $-\frac{1}{\lambda_i}\log U$. Si es la última etapa devuelve X, y si no genera otro U y devuelve X cuando $U > \alpha_i$; en caso contrario pasa a la siguiente etapa. Se usaron $k = 3$, $\lambda = (1, 2, 3)$ y $\alpha = (0.8,\ 0.6)$.

```
def gen_coxian(lams, alphas):
    k = len(lams)
    X = 0.0
    for i in range(k):
        X += -math.log(nextU()) / lams[i]
        if i == k - 1:
            return X, i + 1
        if nextU() > alphas[i]:
            return X, i + 1
```

Listing 7: Variable de Coxian

Con 1000 valores la media muestral fue 1.5951 frente a una teórica de 1.5600, y los primeros valores fueron 0.5051, 1.5197, 3.7064, 0.4867 y 0.2681. La tabla 2 muestra en qué etapa terminó el trabajo, y la figura 6 la distribución obtenida.

	Etapa 1	Etapa 2	Etapa 3
Proporción teórica	0.200	0.320	0.480
Proporción obtenida	0.194	0.304	0.502

Tabla 2: Etapa en la que terminó el trabajo.

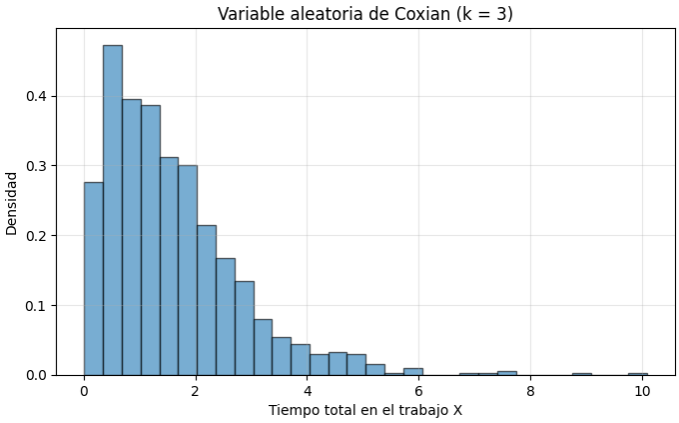


Figura 6: Variable aleatoria de Coxian.

5.3. Generando procesos de Poisson no homogéneos

La intensidad es $\lambda(t) = t/5$ para $0 < t < 5$ y $\lambda(t) = 1 + 5(t - 5)$ para $5 < t < 10$ (figura 7). Su máximo es $\lambda(10) = 26$ y el número esperado de eventos en $[0, 10]$ es 70.

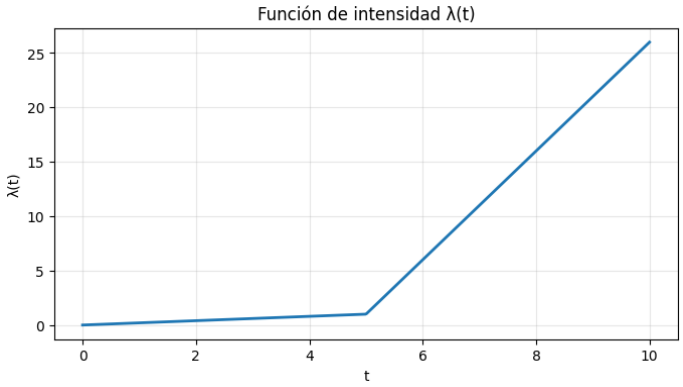


Figura 7: Función de intensidad $\lambda(t)$.

Se implementaron dos algoritmos. El primero es el adelgazamiento con cota fija $\lambda = 26$, que acepta cada candidato con probabilidad $\lambda(t)/26$. El segundo es el adelgazamiento por subintervalos, con cotas 1, 6, 11, 16, 21 y 26 en los tramos $[0, 5)$, $[5, 6)$, $[6, 7)$, $[7, 8)$, $[8, 9)$ y $[9, 10]$.

```
def pnh_thinning(lam_func, lam_max, T):
    t, I, S, candidatos = 0.0, 0, [], 0
    while True:
        t = t - math.log(nextU()) / lam_max
        if t > T:
            break
        candidatos += 1
        if nextU() <= lam_func(t) / lam_max:
            I += 1
            S.append(t)
    return I, S, candidatos
```

Listing 8: Adelgazamiento con cota fija

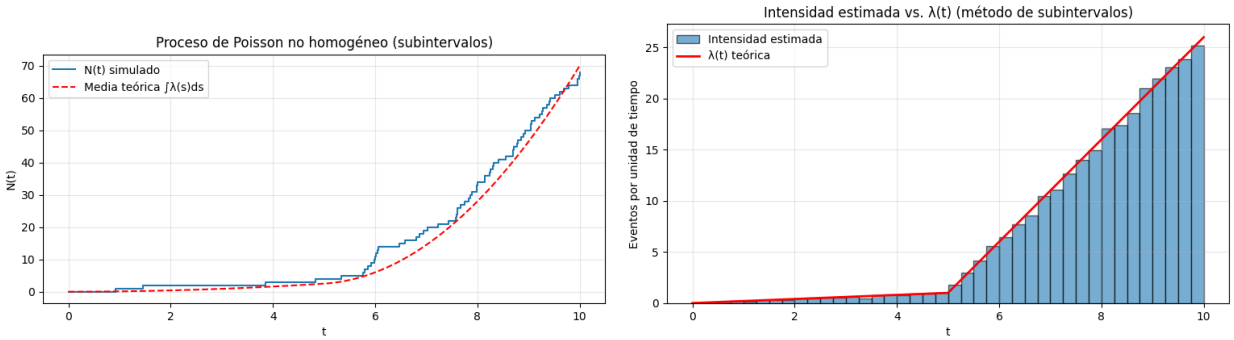
```
def pnh_subintervalos(lam_func, tj, lamj):
    k1 = len(tj)
    t, J, I, S, candidatos = 0.0, 0, 0, [], 0
    X = -math.log(nextU()) / lamj[J]
    while True:
        if t + X > tj[J]:
            if J == k1 - 1:
                break
            X = (X - tj[J] + t) * lamj[J] / lamj[J + 1]
            t = tj[J]
            J += 1
            continue
        t = t + X
        candidatos += 1
        if nextU() <= lam_func(t) / lamj[J]:
            I += 1
            S.append(t)
        X = -math.log(nextU()) / lamj[J]
    return I, S, candidatos
```

Listing 9: Adelgazamiento por subintervalos

En una corrida, la cota fija generó 277 candidatos y aceptó 88 eventos, mientras que los subintervalos generaron 81 candidatos y aceptaron 68. Al repetir cada método 500 veces se obtuvo la tabla 3. Ambos promedian cerca de los 70 eventos esperados, pero los subintervalos necesitan muchos menos candidatos. La figura 8 muestra una realización y la intensidad estimada frente a $\lambda(t)$.

Método	Media de $N(10)$	Candidatos promedio	Aceptación
Cota fija $\lambda = 26$	69.974	259.38	0.270
Subintervalos	69.550	84.56	0.822

Tabla 3: Resultados en 500 réplicas (valor esperado 70).



(a) Realización de $N(t)$

(b) Intensidad estimada

Figura 8: Proceso de Poisson no homogéneo por subintervalos.

Conclusiones

El generador congruencial mixto con $X_0 = 2391$, $a = 25214903917$, $c = 11$ y $m = 2^{48}$ pasó las pruebas de uniformidad y aleatoriedad, por lo que sirvió como base confiable para todo el laboratorio.

Con la transformada inversa se generaron correctamente las cuatro distribuciones de la sección 5.1. Los histogramas siguieron la densidad teórica y las medias muestrales quedaron muy cerca de las teóricas. El método del rechazo también funcionó en los literales a y b, y el promedio de iteraciones (1.6030 y 2.0110) coincidió con la constante c de cada caso.

En el proceso de Poisson el promedio de eventos en 1000 réplicas fue 19.934, prácticamente el valor esperado de 20. La variable de Coxian dio una media de 1.5951 frente a 1.56 teórico, y sus proporciones por etapa quedaron cerca de las teóricas. En el proceso no homogéneo los dos métodos promediaron cerca de los 70 eventos esperados, pero el adelgazamiento por subintervalos necesitó 84.56 candidatos contra 259.38 de la cota fija. Por eso resultó ser el método más eficiente.